

Outputs of Assignment

Create customers Table

```
CREATE TABLE customers AS
SELECT DISTINCT "Customer ID" AS customer_id,
               "Customer Name" AS customer_name,
               Segment
FROM superstore_raw;
```

```
select * from customers;
```

	customer_id	customer_name	Segment
1	CG-12520	Claire Gute	Consumer
2	DV-13045	Darrin Van Huff	Corporate
3	SO-20335	Sean O'Donnell	Consumer
4	BH-11710	Brosina Hoffman	Consumer
5	AA-10480	Andrew Allen	Consumer
6	IM-15070	Irene Maddox	Consumer
7	HP-14815	Harold Pawlan	Home Office
8	PK-19075	Pete Kriz	Consumer

1-500 of 501+ 1-500 of 501+

Create products Table

```
CREATE TABLE products AS
SELECT DISTINCT
  "Product ID" AS product_id,
  "Product Name" AS product_name,
  Category,
  "Sub-Category" AS sub_category
FROM superstore_raw;
```

```
select * from products;
```

	product_id	product_name	Category	sub_category
1	FUR-B0-10001798	Bush Somerset Collection Bookcase	Furniture	Bookcases
2	FUR-CH-10000454	Hon Deluxe Fabric Upholstered Stacking Chairs,...	Furniture	Chairs
3	OFF-LA-10000240	Self-Adhesive Address Labels for Typewriters b...	Office Supplies	Labels
4	FUR-TA-10000577	Bretford CR4500 Series Slim Rectangular Table	Furniture	Tables
5	OFF-ST-10000760	Eldon Fold 'N Roll Cart System	Office Supplies	Storage
6	FUR-FU-10001487	Eldon Expressions Wood and Plastic Desk Access...	Furniture	Furnishings
7	OFF-AR-10002833	Newell 322	Office Supplies	Art
8	TEC-PH-10002275	Mitel 5320 IP Phone VoIP phone	Technology	Phones

1-500 of 501+ 1-500 of 501+

Create orders Table

```
CREATE TABLE orders AS
SELECT
  "Order ID" AS order_id,
  "Customer ID" AS customer_id,
  "Product ID" AS product_id,
  Sales,
  Quantity,
  Profit,
  Region,
  "Order Date" AS order_date
FROM superstore_raw;

select * from orders;
```

	order_id	customer_id	product_id	Sales	Quantity	Profit	Region	order_date
1	CA-2016-152156	CG-12520	FUR-B0-10001798	261.96	2	41.9136	South	11/8/2016
2	CA-2016-152156	CG-12520	FUR-CH-10000454	731.94	3	219.582	South	11/8/2016
3	CA-2016-138688	DV-13045	OFF-LA-10000240	14.62	2	6.8714	West	6/12/2016
4	US-2015-108966	S0-20335	FUR-TA-10000577	957.5775	5	-383.031	South	10/11/2015
5	US-2015-108966	S0-20335	OFF-ST-10000760	22.368	2	2.5164	South	10/11/2015
6	CA-2014-115812	BH-11710	FUR-FU-10001487	48.86	7	14.1694	West	6/9/2014
7	CA-2014-115812	BH-11710	OFF-AR-10002833	7.28	4	1.9656	West	6/9/2014
8	CA-2014-115812	BH-11710	TEC-BH-10002275	907.152	6	90.7152	West	6/9/2014

Step 2: Perform Required Queries

Write and execute SQL queries for each of the following:

1. Find all orders where sales are greater than the average sales. *(Subquery)*

```
SELECT * FROM orders WHERE Sales > ( SELECT AVG(Sales) FROM orders );
```

	order_id	customer_id	product_id	Sales	Quantity	Profit	Region
1	CA-2016-152156	CG-12520	FUR-B0-10001798	261.96	2	41.9136	South
2	CA-2016-152156	CG-12520	FUR-CH-10000454	731.94	3	219.582	South
3	US-2015-108966	S0-20335	FUR-TA-10000577	957.5775	5	-383.031	South
4	CA-2014-115812	BH-11710	TEC-PH-10002275	907.152	6	90.7152	West
5	CA-2014-115812	BH-11710	FUR-TA-10001539	1706.184	9	85.3092	West
6	CA-2014-115812	BH-11710	TEC-PH-10002033	911.424	4	68.3568	West
7	CA-2016-161389	IM-15070	OFF-BI-10003656	407.976	3	132.5922	West
8	CA-2014-105803	DV-13045	OFF-ST-10000760	22.368	2	2.5164	South

2. Find the highest sales order for each customer. *(Subquery)*

```
SELECT * FROM orders o WHERE Sales = ( SELECT MAX(Sales) FROM orders WHERE customer_id = o.customer_id );
```

	order_id	customer_id	product_id	Sales	Quantity	Profit	Region	order_date
1	CA-2016-152156	CG-12520	FUR-CH-10000454	731.94	3	219.582	South	11/8/2016
2	US-2015-108966	S0-20335	FUR-TA-10000577	957.5775	5	-383.031	South	10/11/2015
3	CA-2014-115812	BH-11710	FUR-TA-10001539	1706.184	9	85.3092	West	6/9/2014
4	CA-2015-106320	EB-13870	FUR-TA-10000577	1044.63	3	240.2649	West	9/25/2015
5	US-2015-150630	TB-21520	FUR-B0-10004834	3083.43	7	-1665.0522	East	9/17/2015
6	CA-2016-117590	GH-14485	TEC-PH-10004977	1097.544	7	123.4737	Central	12/8/2016
7	CA-2015-117415	SN-20710	FUR-B0-10002545	532.3992	3	-46.9764	Central	12/27/2015
8	CA-2014-105803	DV-13045	OFF-ST-10000760	22.368	2	2.5164	South	10/11/2015

3. Calculate total sales for each customer. (CTE)

```
WITH customer_sales AS ( SELECT customer_id, SUM(Sales) AS total_sales FROM orders GROUP BY customer_id )
SELECT * FROM customer_sales ORDER BY total_sales DESC;
```

	customer_id	total_sales
1	SM-20320	25043.05
2	TC-20980	19052.218
3	RB-19360	15117.339
4	TA-21385	14595.62
5	AB-10105	14473.571
6	KL-16645	14175.229
7	SC-20095	14142.334
8	HL-15040	12873.297999999999

4. Find customers whose total sales are above average. (CTE+Subquery)

```
WITH customer_sales AS
( SELECT customer_id, SUM(Sales) AS total_sales FROM orders GROUP BY customer_id )
SELECT customer_id, total_sales
FROM customer_sales
WHERE total_sales > ( SELECT AVG(total_sales) FROM customer_sales )
ORDER BY total_sales DESC;
```

	customer_id	total_sales
1	SM-20320	25043.05
2	TC-20980	19052.218
3	RB-19360	15117.339
4	TA-21385	14595.62
5	AB-10105	14473.571
6	KL-16645	14175.229
7	SC-20095	14142.334
8	HL-15040	12873.297999999999

5. Rank all customers based on total sales. (Window Function)

```
WITH customer_sales AS
( SELECT customer_id, SUM(Sales) AS total_sales FROM orders GROUP BY customer_id )
SELECT customer_id, total_sales, RANK() OVER (ORDER BY total_sales DESC) AS sales_rank
FROM customer_sales ORDER BY sales_rank;
```

	customer_id	total_sales	sales_rank
1	SM-20320	25043.05	1
2	TC-20980	19052.218	2
3	RB-19360	15117.339	3
4	TA-21385	14595.62	4
5	AB-10105	14473.571	5
6	KL-16645	14175.229	6
7	SC-20095	14142.334	7
8	HL-15040	12873.297999999999	8

6. Assign row numbers to each order within a customer. (*Window Function + PARTITION BY*)

```
SELECT customer_id, order_id, Sales, ROW_NUMBER() OVER
      ( PARTITION BY customer_id ORDER BY Sales DESC ) AS row_num
FROM orders ORDER BY customer_id, row_num;
```

	customer_id	order_id	Sales	row_num
1	AA-10315	CA-2016-103982	3930.072	1
2	AA-10315	CA-2014-128055	673.568	2
3	AA-10315	CA-2016-103982	431.976	3
4	AA-10315	CA-2017-147039	362.94	4
5	AA-10315	CA-2014-128055	52.98	5
6	AA-10315	CA-2016-103982	41.72	6
7	AA-10315	CA-2015-121391	26.96	7
8	AA-10315	CA-2014-138100	14.94	8

7. Display top 3 customers based on total sales. (*Window Function*)

```
WITH customer_sales AS
  ( SELECT customer_id, SUM(Sales) AS total_sales FROM orders GROUP BY customer_id ),
ranked_customers AS
  ( SELECT customer_id, total_sales, RANK() OVER (ORDER BY total_sales DESC) AS sales_rank FROM customer_sales )
SELECT customer_id, total_sales, sales_rank
FROM ranked_customers WHERE sales_rank <= 3 ORDER BY sales_rank;
```

	customer_id	total_sales	sales_rank
1	SM-20320	25043.05	1
2	TC-20980	19052.218	2
3	RB-19360	15117.339	3

Step 3: Final Combined Query

Write one final query that shows:

- Customer Name
- Total Sales
- Rank

```
WITH customer_sales AS
  ( SELECT customer_id, SUM(Sales) AS total_sales FROM orders GROUP BY customer_id )
SELECT c.customer_name, cs.total_sales, RANK() OVER ( ORDER BY cs.total_sales DESC ) AS customer_rank
FROM customer_sales cs
JOIN customers c ON cs.customer_id = c.customer_id
ORDER BY customer_rank;
```

	customer_name	total_sales	customer_rank
1	Sean Miller	25043.05	1
2	Tamara Chand	19052.218	2
3	Raymond Buch	15117.339	3
4	Tom Ashbrook	14595.62	4
5	Adrian Barton	14473.571	5
6	Ken Lonsdale	14175.229	6
7	Sanjit Chand	14142.334	7
8	Hunter Lopez	12873.297999999999	8

Mini Project: Customer Sales Insights

Answer the following using SQL:

1. Who are the top 5 customers?

```
WITH customer_sales AS
  ( SELECT customer_id, SUM(Sales) AS total_sales FROM orders GROUP BY customer_id )
SELECT c.customer_name, cs.total_sales
FROM customer_sales cs JOIN customers c ON cs.customer_id = c.customer_id
ORDER BY cs.total_sales DESC LIMIT 5;
```

	customer_name	total_sales
1	Sean Miller	25043.05
2	Tamara Chand	19052.218
3	Raymond Buch	15117.339
4	Tom Ashbrook	14595.62
5	Adrian Barton	14473.571

2. Who are the bottom 5 customers?

```

WITH customer_sales AS
  ( SELECT customer_id, SUM(Sales) AS total_sales FROM orders GROUP BY customer_id )
SELECT c.customer_name, cs.total_sales
FROM customer_sales cs JOIN customers c ON cs.customer_id = c.customer_id
ORDER BY cs.total_sales ASC LIMIT 5;

```

	customer_name	total_sales
1	Thais Sissman	4.833
2	Lela Donovan	5.304
3	Carl Jackson	16.52
4	Mitch Gastineau	16.739
5	Roy Skaria	22.328

3. Which customers made only one order?

```

SELECT c.customer_name, COUNT(o.order_id) AS order_count FROM orders o JOIN customers c
ON o.customer_id = c.customer_id
GROUP BY o.customer_id, c.customer_name
HAVING COUNT(o.order_id) = 1;

```

	customer_name	order_count
1	Anthony O'Donnell	1
2	Carl Jackson	1
3	Jocasta Rupert	1
4	Lela Donovan	1
5	Ricardo Emerson	1

5 rows

4. Which customer have above – average sales?

```

WITH customer_sales AS
  ( SELECT customer_id, SUM(Sales) AS total_sales FROM orders GROUP BY customer_id )
SELECT c.customer_name, cs.total_sales, AVG(cs.total_sales) OVER() AS avg_customer_sales
FROM customer_sales cs JOIN customers c
ON cs.customer_id = c.customer_id
WHERE cs.total_sales >
  ( SELECT AVG(total_sales) FROM customer_sales )
ORDER BY cs.total_sales DESC;

```

	customer_name	total_sales	avg_customer_sales
1	Sean Miller	25043.05	5362.36812244898
2	Tamara Chand	19052.218	5362.36812244898
3	Raymond Buch	15117.339	5362.36812244898
4	Tom Ashbrook	14595.62	5362.36812244898
5	Adrian Barton	14473.571	5362.36812244898
6	Ken Lonsdale	14175.229	5362.36812244898
7	Sanjit Chand	14142.334	5362.36812244898
8	Hunter Lopez	12873.297999999999	5362.36812244898

294 rows

5. What is the highest order value per customer?

```
SELECT c.customer_name, MAX(o.Sales) AS highest_order_value FROM orders o JOIN customers c  
ON o.customer_id = c.customer_id GROUP BY o.customer_id, c.customer_name  
ORDER BY highest_order_value DESC;
```

	customer_name	highest_order_value
1	Sean Miller	22638.48
2	Tamara Chand	17499.95
3	Raymond Buch	13999.96
4	Tom Ashbrook	11199.968
5	Hunter Lopez	10499.97
6	Adrian Barton	9892.74
7	Sanjit Chand	9449.95
8	Bill Shonely	9099.93

1-500 of 501+ | Refresh |